Développement mobile - Android

TURLEQUE Gabriel / MOAL Elouan

Rapport de Projet : LimArgent

Application Android de Gestion Financière

Année universitaire 2025 - 2026

Sommaire

Introduction **2**

1 Choix d'Architecture **2**

2 Fonctionnalités Implémentées **3**
2.1 Authentification et Sessions 3
2.2 Gestion des Mouvements et Récurrences 3
2.3 Objectifs d'Épargne 4
2.4 Analyse Graphique (Analytics) 4
2.5 Configuration et Préférences 4
2.6 Mode Hors Ligne et Synchronisation des donnée 5

3 Bibliothèques **5**

4 Utilisation de l'IA **5**

5 Conclusion **6**

6 Test de l'application **6**

Introduction

Ce rapport présente nos choix d'architecture, les bibliothèques utilisées et les fonctionnalités durant ce projet. L'objectif, au-delà de la création d'une application, a été de comprendre les intérêts d'avoir une bonne architecture ainsi que de la respecter du début à la fin.

Dans un second temps, nous avons pu découvrir le monde d'Android et notamment la programmation d'interface (avec Jetpack Compose), la gestion des données avec Firebase, mais aussi la gestion des processus asynchrones grâce aux coroutines Kotlin (et au Worker mais ca ne marchait pas commen on voulait).

La suite du rapport s'articule autour de plusieurs axes. Nous détaillons d'abord nos choix d'architecture, puis nous abordons les différentes fonctionnalité intégrées dans l'application. Nous finirons par par évoqué les bibliothèque que nous avons utilisé et de notre rapport à l'IA durant ce projet.

1. Choix d'Architecture

Pour ce projet nous avons fait le choix d'adopter une **Clean Architecture** qui est donc en couche, à cela nous avons également ajouté une architecture **View-ViewModel** (View est représenté par les fichiers Screen).

Voici une courte description du contenant de nos couches :

— **Couche Domain :** Elle contient les modèles de données purs (les Entités telles que `MoneyMovement`, `ObjectifEpargne`, `User`) et les *Repository*. Autrement dit, les codes qui n'ont pas de logique métier.

— **Couche Application :** Cette couche contient de la logique métier et notamment elle sert de passerelle entre l'UI (couche présentation) et les données (couche data). Au travers des *Services* (ex : `MouvementService`) elle orchestre les règles de gestion (calculs, vérification de soldes, récurrences), et permet aussi de vérifier les données entrées par l'utilisateur pour avoir le moins de logique métier dans l'UI et avoir davantage de sécurité.

— **Couche Data :** Elle est chargée de l'implémentation concrète des repositories de la couche domaine avec les *Providers* (ex : `MouvementProvider`). Ces Providers sont des classes qui communiquent avec la base de donnée. Autrement dit c'est le CRUD, car on va ajouter/supprimer/modifier/mettre à jour/lire les données via les providers qui sont dans cette couche.

— **Couche Presentation (UI) :** Elle contient les *ViewModels* et les vues (Screens) développées en Jetpack Compose. Les Vues se contentent d'afficher les données en observant les états (via des`StateFlow`) exposés par le ViewModel, qui fait lui-même appel à la couche Application.

*Note : Vous noterez peut-être que je n'ai pas mentionner d'Activity. En effet nous avons décider de faire l'application en **Single-Activity**. Au début nous avions bien une Activity par affichage qui se transmettaient un intent avec le context. Sauf que cela nous a poser problème lors de la création de notre bar de navigation, car à chaque changement d'activité l'écran était update et du coup la transition d'une activité à l'autre ne rendait pas bien. Nous avons donc retiré toutes les activités et garder seulement le main activity, qui au lieu d'appelé des activité, appelle des vues.*

Pourquoi avoir choisi la Clean Architecture ? Dès le début du projet, nous avons mis un point d'honneur à soigner notre architecture pour gagner en efficacité. Cela nous a permis de gagner en rigueur et de gagner du temps : la séparation du code nous a permis, à chaque

bug, de cibler le fichier responsable rapidement (puisque l'on savait où chercher). Par ailleurs, ce fonctionnement en couches distinctes nous a offert un contrôle total sur le flux de nos données. En faisant transiter les informations par des étapes bien définies (de la base de données vers l'interface, en passant par notre logique métier), nous avons pu traiter, valider et sécuriser les données avant même qu'elles ne soient affichées à l'utilisateur. Enfin, cette approche nous a fait prendre conscience de l'importance de concevoir une application évolutive et maintenable.

2. Fonctionnalités Implémentées

Voici les différentes fonctionnalités que nous avons implémenté.

2.1. Authentification et Sessions

Description : Afin de garantir la confidentialité des données, chaque utilisateur possède son propre espace. Il peut s'inscrire ou se connecter via un identifiant et un mot de passe.

Comment ça marche :

— **Connexion et Inscription :** L'utilisateur ne saisit qu'un identifiant, que les `LoginService` et `RegisterService` formatent automatiquement pour Firebase Auth. La conformité des mots de passe est également vérifiée dès l'inscription avec un nombre minimum de caractères. L'utilisateur peut modifier un mouvement simplement en cliquant sur ce mouvement dans l'historique de mouvement (dans le menu de modification il peut le supprimer).

— **Profil par défaut :** Lorsqu'un compte est créé, l'application génère automatiquement un document utilisateur dans Firestore. Ce dernier contient des catégories de base (Alimentation, Logement...) pour permettre que l'utilisitateur comprenne comment s'en servir.

— **Session :** Le `SessionManager` écoute l'état de connexion en arrière-plan et met à jour un `StateFlow`. Le `MainActivity` observe ce flux pour rediriger automatiquemen (via `Crossfade`) l'utilisateur vers l'accueil ou l'écran de connexion.

2.2. Gestion des Mouvements et Récurrences

Description : Ajout, modification, suppression et consultation de l'historique des revenus et dépenses dans le menu Home, avec possibilité de rendre un mouvement récurrent.

Comment ça marche :

— **Saisie et Affichage :** Un *Floating action button* permet d'ouvrir un formulaires dans lequel l'utilisateur peut entrer les informations et enregistré le mouvement sur la base Firestore. L'écran d'accueil observe ces données en temps réel via un `StateFlow`, ce qui permet d'appliquer des filtres de recherche (par date, catégorie ou type...) de manière instantanée, sans nouvelle requête vers la BDD.

— **Automatisation :** Si l'utilisateur coche l'option "Récurrent", le système calcule la date de la prochaine exécution du mouvement (suivant la fréquence choisie : quotidien, hebdomadaire, mensuel, etc.). À chaque lancement de l'application, lors de l'initialisation de l'écran d'accueil, le système vérifie automatiquement si des dates d'exécution sont échues. Si une ou plusieurs transactions récurrentes sont en attente, l'application génère instantanément les nouveaux mouvements correspondants et met à jour les prochaines dates de mouvement en base de données avant le chargement de l'historique.

Note : Nous avons tenté de faire ce système de récurrence de manière asynchrone pour pouvoir actualiser les mouvements une fois par jours via un système de ***Worker*** *qui fonctionne*

même lorsque l'application est fermée sauf que le système de gestion de la batterie faisais que la récurrence était vérifiée de manière trop aléatoire, nous avons vu aussi la possibilité d'automatisé cela via Firebase en faisant un programme JavaScript mais c'était payant.

2.3. Objectifs d'Épargne

Description : Définition, suivi et simulation d'objectif d'épargne.

Comment ça marche :

— **Modèle :** Côté métier, la classe`ObjectifEpargne` intègre la logique de calcul sous forme de *getters*. Elle détermine le pourcentage d'accomplissement de l'objectif, son état d'accomplissement et l'estimation des intérêts générés si un taux a été défini.

— **Stockage et Interface :** Les objectifs sont sauvegardés dans une sous-collection Firestore dédiée via l'`ObjectifProvider`. L'interface s'appuie sur le composant `LinearProgressIndicator` de Jetpack Compose pour illustrer la progression, et se met à jour à chaque nouvel apport de l'utilisateur.

2.4. Analyse Graphique (Analytics)

Description : Visualisation des flux financiers de l'utilisateur pour comprendre la répartition de ses dépenses et revenus.

Comment ça marche :

— **Graphiques :** L'API `Canvas` de Jetpack Compose est utilisée pour tracer le graphique en PieChart, avec des arcs de cercle animés via `Animatable`. En revanche, pour la courbe d'évolution du solde, nous avons intégré la bibliothèque **Vico**. Cela nous permet de gérer les axes, et l'adaptation automatique au thème (clair/sombre) via Material 3. Pourquoi utilisé deux façon différentes ? dans un premier temps nous n'avions pas connaissance de Vico et nous Vico ne fait pas de PieChart.

— **Gestion des données :** Les données brutes sont filtrées et regroupées en amont (par l'`AnalyticsViewModel` et calculé par le `BudgetManager`) selon les catégories ou les types de mouvements (entrées/sorties). Ces données calculées sont ensuite transmises au Screen. L'utilisateur a la possibilité de cliquer sur une catégorie parmi lesquelles sont affichées dans le PieChart ce qui lui montre un nouveau PieChart cette fois filtré par titre de mouvement appartenant à la catégorie.

2.5. Configuration et Préférences

Description : Personnalisation de l'application, incluant la gestion des listes de catégories, la configuration des alertes de budget, et la recherche dans l'historique de mouvement.

Comment ça marche :

— **Synchronisation :** Les paramètres de l'utilisateur (seuil d'alerte, catégories personnalisées) sont enregistrés directement dans son document Firestore. Cela permet une sauvegarde des préférences et permet de retrouver l'intégralité de sa configuration lors d'une connexion depuis un autre appareil.

— **Alertes de Budget :** Le système intègre un mécanisme de vérification préventive. Le `HomeViewModel` surveille le solde global en continu : si une nouvelle dépense fait chuter le budget sous le seuil critique configuré dans les préférences, l'application déclenche instantanément une alerte visuelle (Toast) pour avertir l'utilisateur.

— **Thème :** Plusieurs possibilité s'offre à l'utilisateur : soit il ne touche pas à ses préférence alors le thème de l'application correspond au thème système, et s'il coche le théme alors le thème sera sombre et enregistré dans ses préférence, s'il décoche le thème sera lumineux et enregistrer dans ses préférences.

— **Filtrage (dans Home) :** L'application embarque un système de filtres avancés permettant de retrouver facilement une transaction . Le `HomeViewModel` gère cet état localement : l'utilisateur peut faire une recherche textuelle avec des filtres par type (Entrée/Sortie), catégorie, mois et année. Ce tri s'applique sur les données chargées via le `StateFlow`.

2.6. Mode Hors Ligne et Synchronisation des donnée

Description : L'application marche en ligne comme hors ligne, permettant à l'utilisateur de gérer ses finances sans connexion internet.

Comment ça marche :

— **Cache Local (Offline) :** L'application utilise le système de persistance locale de Firebase. Lorsqu'il n'y a pas de réseau, les requêtes de nos `Providers` interrogent automatiquement le cache du téléphone. L'utilisateur peut ainsi consulter son historique, ajouter de nouvelles transactions ou modifier ses objectifs sans aucun blocage de l'interface cependant s'il se déconnecte de son compte alors il ne peut pas se reconnecter.

— **Synchronisation (retour de Offline à Online) :** Dès que l'appareil retrouve une connexion internet, Firebase détecte les modifications effectuées localement et les synchronise en arrière-plan avec la BDD.

3. Bibliothèques

Voici les différentes bibliothèque utilisées dans le projet :

— **Jetpack Compose :** Utilisé pour concevoir l'intégralité de l'interface utilisateur.

— **Dagger Hilt :** Utilisé pour l'injection des dépendances.

— **Firebase (Auth & Firestore) :** Utilisé comme backend.

— **Kotlin Coroutines & Flow :** Utilisés pour l'asynchronisme.

— **Vico :** Utilisée pour la création du graphique linéaire d'évolution du solde.

4. Utilisation de l'IA

Dans le cadre de ce projet, l'IA a joué un rôle d'assistant et de tuteur, et non comme l'auteure de notre code. Face à notre découverte de Kotlin et d'Android, elle nous a permis de découvrir cela plus rapidement notamment en nous fournissant des documentation plus spécifique à nos problèmes que la documentation officiel. Elle s'est révélée très util pour appréhender les concepts compliqués pour nous tels que Jetpack Compose pour l'interface visuelle, ou encore l'intégration de Firebase (plus le côté communication avec la BDD que la mise en place) et Dagger Hilt. De plus, l'IA s'est avérée être un excellent outil de résolution de problèmes : quand nous restions trop longtemps face à un bug elle nous aidait à le résoudre, et nous avons pu apprendre grâce à elle à utiliser des outils comme LogCat pour traquer les bugs invisibles à la compilation mais présents sur le téléphone.

5. Conclusion

Pour conclure, ce que l'on retient principalement de ce projet est l'importance d'avoir une bonne architecture. Mettre en place une structure claire et modulaire nous a permis d'assimiler plus facilement les standard de dev tel que la gestion des flux de données en temps réel et la programmation réactive. Finalement, cette exigence de conception nous a permis se gagner beaucoup de temps sur le projet mais aussi d'avoir plus de facilités à le réalisé et surtout de développé notre rigueur.

6. Test de l'application

Afin que vous puissiez tester notre application nous avons mis en place un compte de test (spécialement prévu pour monsieur Lenoir), permettant d'avoir un aperçu concret de l'utilisation de cette application.
Voici les identifiants de connexion :

— Identifiant : **test**

— Mot de passe : **123456789**